

Here is the complete architectural layout and wiring guide for your ESP32 RC Car.

1. Power Distribution Network

Your system uses a 12V source, split into two separate 5V domains to isolate the logic components from the noisy actuators.

- **12V Main Power:** Connect the 12V Battery positive terminal to the **L298N 12V Input** and the **MP1495 Buck Converter IN+**.
- **Common Ground:** Connect the Battery Ground, L298N Ground, MP1495 IN-, MP1495 OUT-, and **ESP32 GND** together. *All components must share a common ground.*
- **Logic 5V (MP1495 OUT+):** Connect this to the **ESP32 5V/VIN pin**, **HC-SR04 VCC**, **4x IR Sensors VCC**, and **SG90 Servo VCC**.
- **Actuator 5V (L298N 5V Output):** Connect this to the positive power rails of your **Front Lights**, **Decorative Lights**, and **Buzzer** to keep their current draw off the ESP32 and buck converter.

2. Motor Driver & Motors (L298N)

The L298N controls the four 6V-9V gear motors. The left-side motors are wired in parallel, as are the right-side motors.

- **OUT1 & OUT2:** Connect to the two Left Motors.
- **OUT3 & OUT4:** Connect to the two Right Motors.
- **ENA:** Connect to ESP32 **Pin 25**.
- **IN1:** Connect to ESP32 **Pin 26**.
- **IN2:** Connect to ESP32 **Pin 27**.
- **IN3:** Connect to ESP32 **Pin 23**.
- **IN4:** Connect to ESP32 **Pin 13**.
- **ENB:** Connect to ESP32 **Pin 33**.

3. Transistor Circuits (BC547 NPN)

Since the ESP32 pins cannot supply enough current for the lights or buzzer, and you are using a separate 5V source from the L298N, you will use BC547 NPN transistors as low-side switches.

- **Buzzer Circuit:**
 - ESP32 **Pin 19** -> 1kΩ Resistor -> BC547 Base.
 - BC547 Emitter -> Common Ground.
 - BC547 Collector -> Buzzer Negative (-) pin.
 - Buzzer Positive (+) pin -> L298N 5V Output.
- **Front Lights (2 LEDs in parallel):**
 - ESP32 **Pin 17** -> 1kΩ Resistor -> BC547 Base.
 - BC547 Emitter -> Common Ground.
 - BC547 Collector -> Front LEDs Cathodes (-).
 - Front LEDs Anodes (+) -> Current Limiting Resistors (e.g., 220Ω) -> L298N 5V

Output.

- **Decorative Lights (2 LEDs in parallel):**
 - ESP32 **Pin2** -> 1kΩ Resistor -> BC547 Base.
 - BC547 Emitter -> Common Ground.
 - BC547 Collector -> Decorative LEDs Cathodes (-).
 - Decorative LEDs Anodes (+) -> Current Limiting Resistors (e.g., 220Ω) -> L298N 5V Output.

4. Sensors & Servo

- **SG90 Servo Motor:**
 - VCC -> MP1495 5V Output.
 - GND -> Common Ground.
 - Signal -> ESP32 **Pin 18**.
- **HC-SR04 Ultrasonic Sensor:**
 - VCC -> MP1495 5V Output.
 - GND -> Common Ground.
 - Trig -> ESP32 **Pin 32**.
 - Echo -> Voltage Divider -> ESP32 **Pin 35**.
 - **Voltage Divider for Echo:** The HC-SR04 outputs a 5V signal, which will damage the 3.3V ESP32. Use a resistor divider using the formula $V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2}$. Connect a 1kΩ resistor (R_1) between the Echo pin and ESP32 Pin 35, and a 2kΩ resistor (R_2) between ESP32 Pin 35 and Ground.
- **4x FC-51 / HW-201 IR Sensors:**
 - All VCC pins -> MP1495 5V Output.
 - All GND pins -> Common Ground.
 - Left Front OUT -> ESP32 **Pin 34**.
 - Right Front OUT -> ESP32 **Pin 39**.
 - Left Back OUT -> ESP32 **Pin 36**.
 - Right Back OUT -> ESP32 **Pin 4**.

5. Quick Pin Mapping Summary for ESP32 (30-Pin)

ESP32 Pin	Component Connection	Component Type
VIN / 5V	MP1495 Buck Converter 5V Out	Power
GND	System Common Ground	Power
Pin 25	L298N ENA (Left Motor Speed)	Output
Pin 26	L298N IN1 (Left Motor Dir 1)	Output
Pin 27	L298N IN2 (Left Motor Dir 2)	Output
Pin 33	L298N ENB (Right Motor Speed)	Output
Pin 23	L298N IN3 (Right Motor Dir 1)	Output
Pin 13	L298N IN4 (Right Motor Dir 2)	Output
Pin 32	HC-SR04 Trig Pin	Output
Pin 35	HC-SR04 Echo Pin (via Voltage Divider)	Input

ESP32 Pin	Component Connection	
Pin 34	Front Left IR Sensor	
Pin 39	Front Right IR Sensor	
Pin 36	Back Left IR Sensor	
Pin 4	Back Right IR Sensor	
Pin 18	BC547 Base (Siren Control) Output	
Pin 19	Output	
Pin 17	BC547 Base (Front Lights Control)	
Pin 2	BC547 Base (Decorative Lights Control)	Output
Pin 21	OLED SDA (If attached later)	I2C
Pin 22	OLED SCL (If attached later)	I2C